

企业级 AI Agent 应用商店

项目总纲

V0 先备阶段 – 智能体分工、技术选型与文档体系

PM AI: 万能产品小助手 (OpenClaw)

审阅: 魏子政

2026-05-06 版本 v0.1

项目名称	企业级 AI Agent 应用商店
文档类型	项目总纲 (V0 先备阶段)
当前版本	v0.1
文档阶段	V0 (先备阶段: 智能体分工 + 整体框架 + 文档体系搭建)
技术栈	Vue 3 + Element Plus + FastAPI + PostgreSQL + MinIO + OpenSearch + JWT + Docker Compose
PM AI	万能产品小助手 (OpenClaw)
开发者	Cursor Agent

摘要

本文档是企业级 AI Agent 应用商店项目的总纲, 面向项目决策者(魏子政)和未来加入的团队成员。内容覆盖四大核心板块:

- 智能体搭配与 Skill 加载机制:** OpenClaw (PM AI) 和 Cursor Agent 的分工、各自 Skill/规则如何加载、两者的本质区别
- 技术选型:** 每项技术的选用原因、与备选方案的对比、技术栈锁定规则
- 文档体系与生命周期:** tex-pdf 格式规范、命名规则、版本迭代、需求变更处理
- 自闭环机制:** 从需求到交付的五个闭环、监控标准、质量保障

当前项目处于 **V0 先备阶段**, 本总纲是 V0 的核心交付物之一。V0 阶段的目标是搭好智能体分工体系和项目框架, V1 阶段开始接入正式业务需求。

目录

I	智能体搭配与 Skill 加载机制	5
1	为什么需要两种智能体	5
2	两种智能体的 Skill 加载机制	5
2.1	OpenClaw 的 Skill 加载	5
2.2	Cursor 的规则加载	5
2.3	两者的本质区别	6
3	分工矩阵	6
3.1	自验收 vs 独立验收	6
4	调用链路详解	7
4.1	CLI 调用参数说明	7
5	OpenClaw 记忆系统	7
6	Skill/规则文件清单	7
6.1	OpenClaw 的 Skill	7
6.2	Cursor 的规则文件	8
II	技术选型	8
7	锁定技术栈总览	8
8	逐项选型原因	8
8.1	前端：Vue 3 + Element Plus	9
8.2	后端：FastAPI	9
8.3	数据库：PostgreSQL 16	9
8.4	搜索引擎：OpenSearch 2.x	10
8.5	对象存储：MinIO	10
8.6	认证授权：JWT + RBAC	10
8.7	容器化：Docker + Docker Compose	10
9	技术栈演进记录	10
III	文档体系与生命周期	10

10 文档目录结构	11
10.1 文档分类与维护责任	11
11 tex-pdf 文档规范	11
11.1 格式要求	12
11.2 文件命名规范	12
11.3 编译命令	12
11.4 已知限制	12
12 文档版本管理	13
12.1 版本号规则	13
12.2 版本阶段定义	13
12.3 变更日志	13
13 需求新增处理流程	13
13.1 需求来源	13
13.2 5 步处理流程	14
13.3 优先级定义	14
13.4 需求变更控制	14
IV 自闭环机制	14
14 自闭环定义	14
15 五个闭环	15
15.1 闭环 1: 需求闭环	15
15.2 闭环 2: 开发闭环	15
15.3 闭环 3: 测试闭环	15
15.4 闭环 4: 文档闭环	15
15.5 闭环 5: 审计闭环	15
16 闭环监控标准	15
V 当前状态与下一步	16
17 V0 阶段已完成项	16
18 V0 阶段待完成项	16
19 V1 阶段规划	17
A 附录 A: Git 分支策略	17

B 附录 B：提交规范	17
C 附录 C：新成员上手阅读顺序	17
D 附录 D：文档索引	17

Part I

智能体搭配与 Skill 加载机制

为什么需要两种智能体

本项目是一个完整的企业级应用，需要两类截然不同的能力：

能力类型	说明	适合的智能体
产品管理	需求理解、任务拆解、质量审计、进度追踪、文档维护、飞书沟通	OpenClaw (PM AI)
代码开发	编写前后端代码、运行终端命令、构建项目、调试修复	Cursor Agent

核心分工链路：魏子政只跟 OpenClaw 对话 → OpenClaw 拆解需求并调度 Cursor → Cursor 写代码 → OpenClaw 审计质量 → 向魏子政汇报。

为什么不能只用一个？

- OpenClaw 是常驻 **AI 助手**：有记忆系统（memory 文件）、有飞书通道、能主动推送、能执行非代码任务
- Cursor 是**代码编辑器内置 AI**：擅长读写代码、运行命令、理解项目上下文，但没有持久记忆和外部通信能力
- 两者互补：OpenClaw 管**脑**（规划/审计/记忆），Cursor 管**手**（编码/构建/自测）

两种智能体的 Skill 加载机制

这是整个协作体系最核心的区别。

OpenClaw 的 Skill 加载

- **触发时机**：OpenClaw 启动会话时，系统提示中包含 <available_skills> 列表
- **加载方式**：收到用户消息后，AI **自主判断**该加载哪个 skill，用 read 工具读取对应的 SKILL.md
- **存放位置**：
 - ~/.openclaw/skills/ – 全局 skills（所有项目共享）
 - workspacenew1/skills/ – 工作区 skills（项目专属）
- **文件格式**：普通 Markdown，文件名固定 SKILL.md

Cursor 的规则加载

- **触发时机**：Cursor 打开项目工作区时自动扫描

- **加载方式:**

- CLAUDE.md - 项目根目录, **每次对话自动加载**, 总入口
- .cursor/rules/*.mdc - MDC 格式, 支持两种触发:
 - * alwaysApply: true → 每次对话都加载
 - * globs: ["backend/**"] → 只在编辑匹配目录的文件时加载

- **文件格式:** MDC (YAML front matter + Markdown)

两者的本质区别

维度	OpenClaw Skill	Cursor Rule
文件位置	skills/xxx/SKILL.md	.cursor/rules/xxx.mdc
给谁用	OpenClaw (PM AI)	Cursor Agent
格式	普通 Markdown	MDC (YAML + MD)
加载触发	AI 自主判断	alwaysApply 或 globs
记忆	有 (memory 文件系统)	无 (每次对话全新)
通信	有 (飞书通道)	无
代码修改	不直接改代码	读写代码、运行命令

关键理解: skills/ 目录下的 SKILL.md 是给 **OpenClaw** 读的 (让 PM 知道规范长什么样), .cursor/rules/ 是给 **Cursor** 读的 (让 Cursor 写代码时自动遵守规范)。两者内容有关联但职责不同。

分工矩阵

职责	OpenClaw (PM)	Cursor Agent
需求生成与拆解	✓	
独立验收	✓	
文档维护与汇报	✓	
进度管理	✓	
UI 设计		✓
前端开发		✓
后端开发		✓
环境搭建与运维		✓
自验收		✓
响应 PM 验收		✓

自验收 vs 独立验收

Cursor 自验收 (每次开发完成后执行):

- 运行 npm run build 确认前端无报错

- 运行后端启动确认服务正常
- 对照 testing.mdc 逐项自检
- **PM 独立验收** (Cursor 返回结果后执行):
 - 检查产出物是否完整覆盖需求
 - 站在用户角度验证功能
 - 执行查证三问 (目标达成? 细节硬伤? 换位质疑?)
 - **不依赖** Cursor 的自测结论

调用链路详解

一次完整的任务执行流程:

1. **魏子政在飞书发消息**: 如「加一个应用评分功能」
2. **OpenClaw 接收**: 加载 pm/SKILL.md, 读取 memory 恢复上下文, 读取项目文档
3. **PM 执行**: 记录任务到 memory, 拆解需求为开发任务, 写入 PROGRESS.md
4. **调度 Cursor**:

```
cursor agent --print --trust \  
  --workspace projects/enterprise-app-store \  
  "实现应用评分功能: 1.数据库加 ratings 表  
  2.后端评分CRUD接口 3.前端评分展示组件"
```

5. **Cursor 执行**: 自动加载 CLAUDE.md + rules/, 编写代码, 返回结果
6. **PM 审计**: 加载 tester/SKILL.md, 检查产出物, 执行查证三问
7. **汇报**: 飞书通知魏子政, 更新 PROGRESS.md

CLI 调用参数说明

```
cursor agent --print --trust --workspace <项目路径> "任务描述"  
# --print      : 输出完整响应 (OpenClaw 可捕获)  
# --trust      : 跳过人工确认 (PM 调度场景)  
# --workspace: 指定项目根目录
```

OpenClaw 记忆系统

Cursor **没有跨会话记忆**。每次对话都是全新的, 只靠 CLAUDE.md 和 rules/ 获取上下文。

OpenClaw **有文件级记忆**:

这就是为什么 OpenClaw 能做 PM 而 Cursor 不能 - Cursor 不知道昨天做了什么、上周改了什么需求、魏子政有什么偏好。OpenClaw 通过文件记忆「知道」这些。

Skill/规则文件清单

OpenClaw 的 Skill

文件	用途	更新频率
SOUL.md	身份、性格、行为准则	很少改
USER.md	用户信息、偏好	偶尔更新
MEMORY.md	长期记忆（项目经验、教训）	每隔几天整理
memory/YYYY-MM-DD.md	当日任务日志	每天
PROGRESS.md	项目进度	每次任务后
BUGS.md	Bug 记录	发现 Bug 时

Skill	文件	用途
PM 产品经理	skills/pm/SKILL.md	需求拆解、任务分配、审计、查证
UI 设计	skills/ui-design/SKILL.md	设计方案输出、视觉规范
测试	skills/tester/SKILL.md	阶段门控验证、功能测试
开发规范	skills/developer/SKILL.md	开发规范参考（PM 审计用）

Cursor 的规则文件

规则文件	触发条件	用途
CLAUDE.md	每次必加载	项目总入口，指向 rules/
project-overview.mdc	alwaysApply	技术栈锁定、禁止事项
backend-rules.mdc	backend/	async/await、Pydantic、错误格式
frontend-rules.mdc	frontend/	<script setup>、TS、Pinia
ui-design.mdc	frontend/	配色、间距、组件规范
api-conventions.mdc	backend/	RESTful 命名、分页、统一响应
testing.mdc	alwaysApply	自测要求
database.mdc	数据库相关	迁移、软删除、审计字段

Part II

技术选型

锁定技术栈总览

硬约束：前端不用 Next.js/React，后端不用 Spring/Java。技术栈一经锁定，OpenClaw 和 Cursor 均不得自行变更。

逐项选型原因

层	技术	用途
前端	Vue 3 + Vite + Element Plus + Pinia + TypeScript	管理后台 + 开发者门户
后端	FastAPI + SQLAlchemy (async) + Pydantic	REST API 服务
数据库	PostgreSQL 16	元数据 + JSONB + 审计日志
对象存储	MinIO (S3 兼容)	应用包存储
搜索	OpenSearch 2.x	全文检索 + 向量语义搜索
认证	JWT (python-jose + passlib)	用户认证 + RBAC
容器化	Docker + Docker Compose	本地开发 + 生产部署
迁移	Alembic	数据库 schema 版本管理

前端：Vue 3 + Element Plus

为什么选 Vue 3 ?

1. 学习曲线低：模板语法直观，实习生上手快，比 React hooks 概念负担轻
2. Element Plus 生态成熟：企业级中后台组件库，表格/表单/弹窗/分页开箱即用，减少 50%+ UI 工作量
3. 国内团队主流：中文文档完善，社区活跃，招人容易
4. TypeScript 一等支持：<script setup> + TS 组合式 API
5. Vite 构建极快：HMR 毫秒级

为什么不用 Next.js/React ? SSR/SSG 对内部管理系统价值不大；React 需额外状态管理方案；团队偏 Python，Vue 模板语法更易跨角色理解。

Element Plus vs Ant Design Vue vs Arco Design：Element Plus 组件最全、文档最好、社区最大，选定。

后端：FastAPI

为什么选 FastAPI ?

1. 异步原生：async/await 一等支持，天然适合 IO 密集型（文件上传、搜索调用、扫描集成）
2. 自动文档：Pydantic 模型自动生成 Swagger/OpenAPI 文档
3. 类型安全：请求/响应自动校验，参数错误自动返回 422
4. Python 生态：与安全扫描工具（Semgrep/ClamAV）、LLM API 天然互通
5. 开发效率高：代码量少，原型快

为什么不用 Spring Boot ? 启动慢、代码量大，对中小团队效率偏低；安全扫描工具 Python 集成更方便。

数据库：PostgreSQL 16

为什么选 PostgreSQL ?

1. 事务完整：ACID 保证，适合应用元数据、权限、审计
2. JSONB 类型：扫描结果是嵌套 JSON，PG JSONB 支持查询和索引
3. 扩展性强：pgvector（向量搜索）、全文检索、范围类型

4. 运维成熟：备份、复制、监控工具链完善

为什么不用 MongoDB ? 去掉 MongoDB 减少组件；扫描结果用 PG JSONB 足够。
为什么不用 MySQL ? JSONB 支持不如 PG 灵活，复杂查询能力偏弱。

搜索引擎：OpenSearch 2.x

为什么选 OpenSearch ?

1. 全文 + 向量一体化：一个集群同时支持关键词搜索和向量语义搜索
2. 权限过滤：DSL 支持 per-document 权限查询
3. 中文分词：IK Analysis 插件
4. Apache 2.0 许可：无 SSPL 限制 (Elasticsearch 7.11+ 改为 SSPL)

对象存储：MinIO

为什么选 MinIO ?

1. S3 API 兼容：随时可切换到 AWS S3/阿里 OSS
2. 私有化部署：数据不出域
3. 预签名 URL：前端直接下载，不经后端转发
4. Docker 部署简单：一个容器搞定

认证授权：JWT + RBAC

为什么选 JWT 而不是 Keycloak ?

1. 轻量：Keycloak 需要 JVM + 数据库 + 独立服务，JWT 只需两个 Python 库
2. 够用：初期只需 4 角色 (admin/reviewer/developer/viewer)
3. 无状态：Token 自包含，不需要每次查库

后续扩展：需要 LDAP/SSO/多租户时引入 Keycloak，JWT 签发逻辑从本地改为对接 Keycloak。

容器化：Docker + Docker Compose

为什么选 Docker Compose 而不是 K8s ?

1. MVP 不需要 K8s 编排/调度/自愈能力
2. 开发体验好：docker compose up 一键启动所有依赖
3. 运维简单：5-10 人团队不需要专门 K8s 运维

后续扩展：需要多实例/灰度发布时迁移 K8s，Dockerfile 不需要改，只需加 Helm Chart。

技术栈演进记录

原则：「够用即可、按需扩展」。后续需要时再逐个引入。

原组件	替代方案	精简原因
Next.js	Vue 3 + Vite	内部管理不需要 SSR
MongoDB	PostgreSQL JSONB	减少组件，JSONB 足够
Keycloak	JWT + RBAC 表	初期不需要完整 IdP
Temporal	数据库状态机	审批流程简单，不需要工作流引擎
Kafka	Python asyncio	初期异步量不大
K8s + Helm	Docker Compose	MVP 单机部署足够
Prometheus + Grafana	延后实现	MVP 优先功能，监控后续补

表 1: v1.0 → v2.0 精简记录

Part III

文档体系与生命周期

文档目录结构

enterprise-app-store/	
reference/	# 参考文档（调研、规范）
DOC-NAMING-CONVENTION.tex/pdf	# 文档命名规范
REF-001-skill-store-arch-survey	# 开源 Skill 商店架构调研
...	# 后续新增 REF-xxx
demands/	# 需求变更文档
DEMAND-001-role-redefine	# 角色分工重新定义
...	# 后续新增 DEMAND-xxx
AGENT-ARCHITECTURE.md	# 智能体搭配架构
tech-selection.md	# 技术选型说明书
TEAM-MANUAL.md	# 团队协作说明书
DOC-LIFECYCLE.md	# 文档生命周期管理
CHANGELOG.md	# 变更日志
PROGRESS.md	# 进度追踪表
BUGS.md	# Bug 记录
CLAUDE.md	# Cursor 总入口
.cursor/rules/*.mdc	# Cursor 编码规则
skills/pm tester ui-design developer/SKILL.md	
database/SCHEMA.md	# 数据库设计

文档分类与维护责任

tex-pdf 文档规范

类别	维护者	文档举例
产品与规范	OpenClaw	AGENT-ARCHITECTURE.md, tech-selection.md, TEAM-MANUAL.md
参考文档 (reference/)	OpenClaw	REF-xxx, DOC-NAMING-CONVENTION
需求文档 (demands/)	OpenClaw	DEMAND-xxx
设计与规范	OpenClaw 设计	SCHEMA.md, skills/*/SKILL.md
Cursor 规则	OpenClaw	CLAUDE.md, .cursor/rules/*.mdc
开发文档	Cursor 输出	BUGS.md

格式要求

- 源文件：.tex 格式，提交 Git 版本控制
- 交付物：.pdf 格式，用于审阅和分发
- 编译工具：Tectonic (~/.local/tectonic/tectonic)
- 管理者：PM AI（万能产品小助手）

文件命名规范

格式：{前缀}-{三位编号}-{kebab-case 简述}.tex/pdf

前缀	含义	目录
REF	参考文档	reference/
DEMAND	需求变更	demands/
SPEC	技术规格	reference/
GUIDE	操作指南	reference/

编译命令

```
cd reference/ && ~/.local/tectonic/tectonic REF-001-xxx.tex
cd demands/ && ~/.local/tectonic/tectonic DEMAND-001-xxx.tex
```

已知限制

1. 避免 fontawesome5 命令 (\faCalendar 不可用)
2. 避免 Unicode 箭头 (用 \to 或 --> 替代)
3. 避免 listings 的 language 参数 (部分语言定义缺失)
4. Overfull hbox 少量警告可接受

文档版本管理

版本号规则

语义化版本：vX.Y.Z

位	含义	示例
X（主版本）	架构/技术栈重大变更	v1.0 → v2.0
Y（次版本）	新增模块/功能	v2.0 → v2.1
Z（补丁）	文档修正、措辞优化	v2.1 → v2.1.1

版本阶段定义

阶段	名称	内容范围
V0	先备阶段	智能体分工、商店整体框架、文档体系、技术选型
V0.x	先备迭代	分工调整、架构调研、参考方案、规范补充
V1	正式需求	第一批正式业务需求接入
V1.x	功能迭代	功能增强、Bug 修复、性能优化
V2+	演进阶段	架构升级、生态扩展、商业运营

当前阶段：V0（先备阶段）。本总纲是 V0 的核心交付物。

变更日志

项目根目录维护统一的 `CHANGELOG.md`，记录所有文档和代码的变更。每个重要文档头部标注版本号、日期、变更摘要。

需求新增处理流程

需求来源

来源	示例
魏子政直接提	「加一个应用评分功能」
阶段审计发现	「缺少数据导出功能」
Bug 修复衍生	「修复后发现需要加校验」

5 步处理流程

1. **需求记录**：写入 memory 文件，包含需求描述、提出者、优先级
2. **影响评估** (OpenClaw 执行):
 - 技术影响：改哪些模块？新增哪些表/接口/页面？
 - 文档影响：哪些文档需要更新？
 - 进度影响：对当前里程碑有无影响？
3. **方案设计** (复杂需求)：更新 SCHEMA.md、补充 SKILL.md、涉及技术栈变更须魏子政审批
4. **任务拆解与分配**：拆解为可独立完成的开发任务，写入 PROGRESS.md
5. **开发--测试--审计--交付**：Cursor 执行 → OpenClaw 审计 → 文档同步 → 汇报

优先级定义

级别	标签	含义	处理时效
P0	阻塞	阻塞当前阶段推进	立即
P1	严重	核心功能缺陷	当天
P2	一般	重要但不紧急	排入当前/下阶段
P3	轻微	锦上添花	有空再做

需求变更控制

小变更 (不影响已有模块)：PM 直接处理，事后说明。

中变更 (影响已有模块但不涉及架构)：PM 评估 → 简要说明 → 执行。

大变更 (涉及架构/技术栈/数据库结构)：PM 评估 → 详细方案 → 魏子政审批 → 执行。

Part IV

自闭环机制

自闭环定义

自闭环 = 从需求提出到交付完成，**所有环节在同一体系内完成**，不需要外部干预。

魏子政提需求

-> OpenClaw 接收 -> 记录 -> 评估
-> 方案设计 -> 文档更新
-> Cursor 开发 -> 代码产出
-> OpenClaw 测试 -> 查证 -> 审计

-> 文档同步更新
-> 交付输出 -> 反馈 -> 循环

五个闭环

闭环 1：需求闭环

- 输入：魏子政提需求
- 过程：记录 → 评估 → 设计 → 拆解 → 分配
- 输出：PROGRESS.md 中的任务条目（附验收标准）
- 闭合标志：PROGRESS.md 出现新任务，状态为「待开始」

闭环 2：开发闭环

- 输入：Cursor 收到任务
- 过程：读 rules/ → 写代码 → 自测 → 提交
- 输出：可运行的代码
- 闭合标志：npm run build 通过 + 后端启动正常

闭环 3：测试闭环

- 输入：Cursor 产出代码
- 过程：OpenClaw 执行测试验证（接口 → 功能 → 流程 → UI）
- 输出：测试报告
- 闭合标志：tester/SKILL.md 验证项全部通过

闭环 4：文档闭环

- 输入：代码/需求/规范变更
- 过程：更新受影响文档 → 生成新 PDF → 记录 CHANGELOG
- 输出：所有文档与代码保持一致
- 闭合标志：CHANGELOG.md 有新记录，文档版本号已更新

闭环 5：审计闭环

- 输入：测试通过的开发任务
- 过程：OpenClaw 执行查证三问（目标达成？细节硬伤？换位质疑？）
- 输出：审计结论（通过/需修正）
- 闭合标志：查证三问全部通过，PROGRESS.md 任务状态变为「已完成」

闭环监控标准

OpenClaw 在每次任务完成后，检查所有 5 个闭环是否闭合：

检查项	闭合标准
需求闭环	PROGRESS.md 有任务记录
开发闭环	Cursor 返回成功结果
测试闭环	tester/SKILL.md 验证项全部通过
文档闭环	受影响文档已更新 + CHANGELOG.md 有记录
审计闭环	查证三问通过

任何闭环未闭合，任务不能标记为「已完成」。

Part V

当前状态与下一步

V0 阶段已完成项

1. 智能体分工定义：OpenClaw = 需求生成 + 拆解 + 独立验收；Cursor = 前后端 + UI + 环境 + 运维 + 测试

2. 技术选型 v2.0 锁定：Vue 3 + FastAPI + PG + MinIO + OpenSearch + JWT + Docker Compose

3. Skill/规则体系搭建：
 - OpenClaw：4 个 SKILL.md (pm、tester、ui-design、developer)
 - Cursor：1 个 CLAUDE.md + 7 个.mdc 规则文件

4. 文档体系搭建：
 - 产品规范：AGENT-ARCHITECTURE.md、tech-selection.md、TEAM-MANUAL.md、DOC-LIFECYCLE.md
 - tex-pdf 规范：DOC-NAMING-CONVENTION（命名规范）、REF-001（架构调研）
 - 需求文档：DEMAND-001（角色分工重新定义）

5. 自闭环机制设计：5 个闭环 + 监控标准

6. 数据库设计：database/SCHEMA.md（ER 图 + 数据字典）

7. 开源方案调研：Flow Nexus App Store（MCP 架构）+ Membrane Marketplacer（中间件代理）

V0 阶段待完成项

1. 强化 ui-design.mdc，补充常用页面模式（列表页、详情页、表单页、审批页）

2. 从 cursor.directory 精选 ally/响应式相关规则片段

3. 评估 Figma MCP 集成可行性（如魏子政提供设计稿）

4. 确认数据库 schema 与技术选型的最终一致性

V1 阶段规划

V1 阶段开始接入正式业务需求，预计里程碑：

阶段	核心交付物	预计周期
环境搭建	Docker Compose + 登录页 + /health	1 周
核心功能	应用 CRUD + 文件上传 + 用户认证	1 周
搜索 + 审批	OpenSearch + 审批流程	1 周
安全扫描	Semgrep + ClamAV + LLM 集成	1 周
打磨 + 部署	UI 优化 + Docker 生产部署	2 周

附录 A：Git 分支策略

main	# 稳定版本，里程碑合并
dev	# 开发分支，日常提交
feat/xxx	# 新功能分支
fix/xxx	# Bug 修复分支

附录 B：提交规范

feat	: 新增应用评分功能
fix	: 修复登录页 token 过期未跳转
docs	: 更新技术选型文档 v2.0
refactor	: 重构上传模块为异步

附录 C：新成员上手阅读顺序

1. TEAM-MANUAL.md - 了解怎么协作
2. AGENT-ARCHITECTURE.md - 了解智能体怎么配合
3. tech-selection.md - 了解技术栈和选型原因
4. database/SCHEMA.md - 了解数据库设计
5. PROGRESS.md - 了解当前进度

附录 D：文档索引

文档	类型	说明
本文件（项目总纲 PDF）	tex-pdf	V0 核心交付物，总纲
AGENT-ARCHITECTURE.md	Markdown	智能体搭配详细说明
tech-selection.md	Markdown	技术选型 v2.0
TEAM-MANUAL.md	Markdown	团队协作说明书
DOC-LIFECYCLE.md	Markdown	文档生命周期管理
CHANGELOG.md	Markdown	变更日志
PROGRESS.md	Markdown	进度追踪表
CLAUDE.md	Markdown	Cursor 总入口
database/SCHEMA.md	Markdown	数据库设计
BUGS.md	Markdown	Bug 记录
reference/DOC-NAMING-CONVENTION	tex-pdf	文档命名规范
reference/REF-001-skill-store-arch-survey	tex-pdf	开源 Skill 商店架构调研
demands/DEMAND-001-role-redefine	tex-pdf	角色分工重新定义